國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 08

姓名：廖彥瑄

學號：10810030

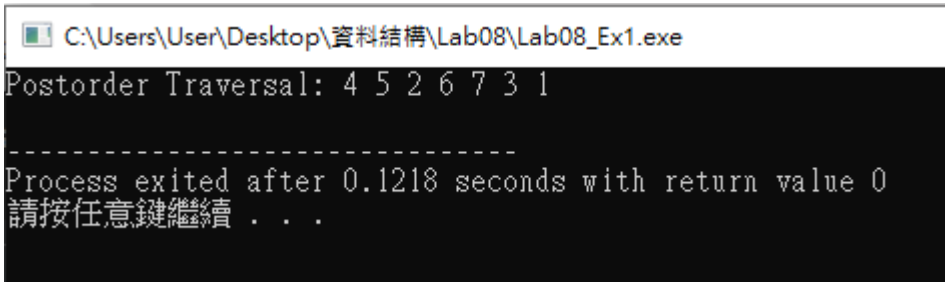
Lab08-Ex1.
Modify the inorder traversal function to a postorder traversal function

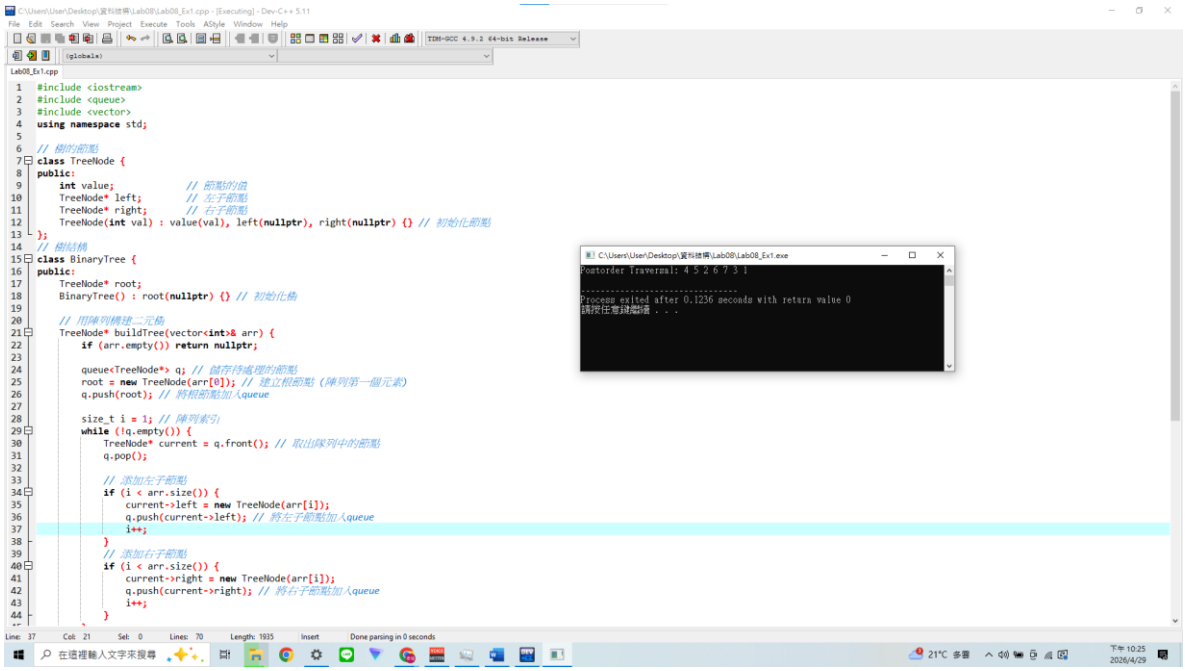
Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
    TreeNode* left;         // 左子節點
    TreeNode* right;        // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};
// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
```

```
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入 queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入 queue
                i++;
            }
        }

        return root;
    }
    void postorderTraversal(TreeNode* node) {
        if (node == nullptr) return;//判斷是不是空的
        postorderTraversal(node->left);//先看左邊
        postorderTraversal(node->right);//再看右邊
        cout << node->value << " ";//最後看原本的
    }

};
int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    cout << "Postorder Traversal: ";
    tree.postorderTraversal(tree.root);
    cout << endl;

    return 0;
}
```

Result





Postorder Traversal: 4 5 2 6 7 3 1

Lab08-Ex2.

Implement a method to find the maximum value in the left and right subtrees.

Code

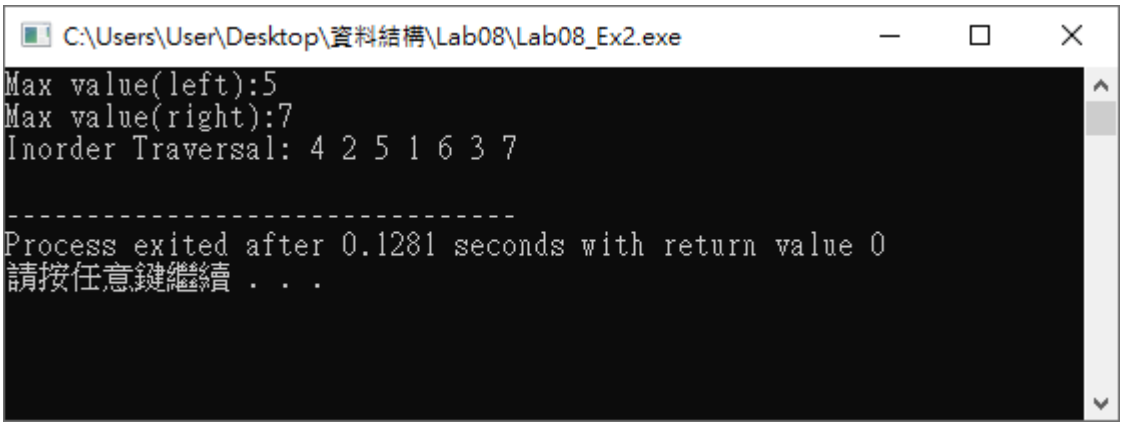
```
#include <iostream>
#include <queue>
#include <vector>
#include <climits>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
```

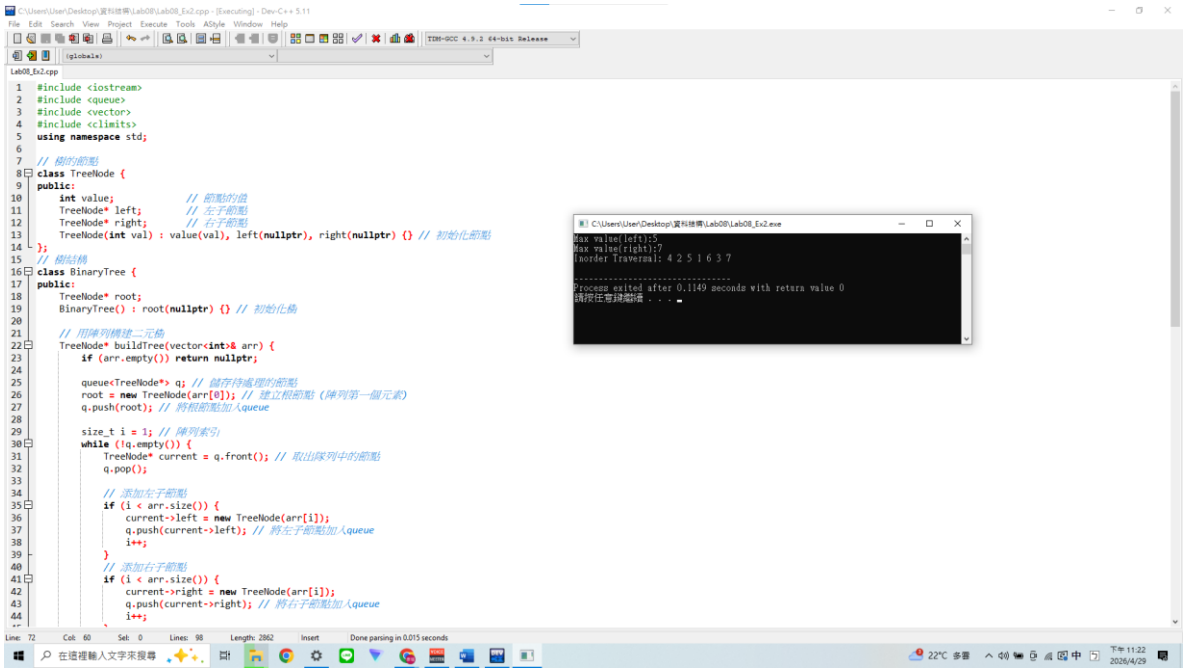
```
    TreeNode* left;        // 左子節點
    TreeNode* right;       // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};
// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入 queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入 queue
                i++;
            }
        }

        return root;
```

```
    }
    // 中序遍歷
    void inorderTraversal(TreeNode* node) {
        if (node == nullptr) return; // 如果節點為空，忽略

        inorderTraversal(node->left);  // 遍歷左子樹
        cout << node->value << " ";    // 訪問當前節點
        inorderTraversal(node->right); // 遍歷右子樹
    }
    int findmax(TreeNode* node) {
        queue<TreeNode*> q;//建立queue
        q.push(node);
        int max = node->value;//假設第一個是最大值
        while (!q.empty()) {
            TreeNode* current = q.front();//找到下一個節點的值
            q.pop();//移除下一個節點的值
            if (current->value > max) {
                max = current->value;//比較誰比較大
            }
            if (current->left != nullptr) {
                q.push(current->left);//把左側節點的值放進queue
            }
            if (current->right != nullptr) {
                q.push(current->right);//把右側節點的值放進queue
            }
        }
        return max;
    }

};
int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹
```

```
    cout<<"Max value(left):"
        <<tree.findmax(tree.root->left)<<endl;
    cout<<"Max value(right):"
        <<tree.findmax(tree.root->right)<<endl;

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    return 0;
}
```

Result





Max value(left):5

Max value(right):7
Inorder Traversal: 4 2 5 1 6 3 7

Discussion

把左側 subtree 和右側 subtree 分別傳進函數，函數裡面用 queue 把整個 subtree 的節點一個一個取出來，藉由比大小找到最大值再回傳

加分題:對於找最大值沒有影響